DeST-Fuzzing: Defense-State Transition Optimization for Stable Jailbreak Discovery in Large Language Models

Anonymous Authors

Abstract

Automated jailbreak fuzzing is an important technique for evaluating the safety robustness of large language models (LLMs). Existing fuzzing methods usually use binary success/failure feedback to guide prompt mutation. Such feedback conflates complete refusals with partial acceptances near the defense boundary, making intermediate progress invisible and reducing the effectiveness of subsequent search. We propose DeST-Fuzzing, a Defense-State Transition optimization framework for stable jailbreak discovery. DeST-Fuzzing first maps target responses into four ordered defense states, from complete refusal to full acceptance, and defines a boundary potential to quantify how far a prompt has progressed toward policy bypass. It then treats mutation operators as state-transition actions and estimates their potential gains under different defense states, enabling transition-aware operator selection. To search over multi-step transition paths, DeST-Fuzzing builds a non-linear template tree with dynamic intermediate expansion, which preserves promising intermediate prompts instead of discarding them after local failures. Finally, an uncertainty-gated value update suppresses unstable branches caused by inconsistent target responses. Experiments on representative commercial LLMs show that DeST-Fuzzing improves attack success rate, query efficiency, and reproducibility over state-of-the-art jailbreak fuzzing baselines.

CCS Concepts

• **Security and privacy → Software security engineering**.

Keywords

Jailbreak Fuzzing, Large Language Model Security, Automated Red Teaming, Stability-Aware Tree Search

ACM Reference Format:

Anonymous Authors. 2026. DeST-Fuzzing: Defense-State Transition Optimization for Stable Jailbreak Discovery in Large Language Models. In . ACM, New York, NY, USA, 17 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Large Language Models (LLMs) have shown strong practical value in natural language understanding and complex reasoning, leading to their integration across diverse domains such as healthcare [35], finance [12], and software engineering [5, 39]. As LLMs are increasingly deployed in productivity tools and information retrieval systems, ensuring their reliability and security has become a key requirement for sustained use in critical infrastructure [4, 29].

Despite recent progress in safety alignment [10, 33, 41], LLMs remain vulnerable to jailbreak attacks, where carefully crafted prompts induce the model to generate content that violates its safety constraints [13, 19]. Such attacks can elicit prohibited outputs, including malicious code, misinformation, hate speech, or instructions for illegal activities. As LLMs become more capable and widely accessible, jailbreak vulnerabilities create a practical risk: a small number of effective prompts can be reused or adapted at scale to produce harmful content. Therefore, systematic jailbreak testing is necessary for identifying latent safety weaknesses and evaluating whether LLM defenses remain robust under evolving adversarial strategies [33, 41].

Security assessments of LLMs against jailbreak attacks have traditionally relied on manual red-teaming, in which human experts craft adversarial inputs to identify model vulnerabilities [6, 31]. These efforts often use linguistic and psychological tactics, such as persona-based role-playing and logical obfuscation, to circumvent safety constraints. Complex narrative prompts, exemplified by the notorious "Do Anything Now (DAN)" variants [33], have been demonstrated to bypass security filters in certain settings [32, 33]. However, manual red-teaming scales poorly because it is labor-intensive and often requires substantial expert time to produce individual successful jailbreak prompts. As a result, purely manual approaches often cannot keep pace with rapid deployment cycles and evolving defense mechanisms in the latest LLMs [8].

To address the scalability limitations of manual red-teaming, recent studies have proposed automated adversarial testing frameworks that leverage LLMs [6, 30, 31, 41]. Many of these methods follow a fuzzing paradigm in which an attacker model iteratively mutates seed templates to generate new prompts. The target model's responses are then used as feedback to guide subsequent mutations. Prior work has explored several variants: (1) employing Monte Carlo tree search to select and mutate templates [41]; (2) framing the attack as an adversarial dialogue in which the attacker refines prompts based on refusal outcomes [7]; and (3) implementing tree-structured search to manage conversational branches and prune ineffective paths [26]. Although these automated methods reduce the cost of manual testing, they often rely on coarse, binary feedback and simple sequential iteration, which can limit their ability to explore complex defense boundaries.

Specifically, two major limitations restrict the performance of existing methods. First, binary feedback provides only success or failure labels, which causes state aliasing near the defense boundary. Prompts that trigger complete refusals and prompts that already elicit partial acceptance are both treated as failures, hiding useful intermediate progress and weakening the feedback signal

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, Washington, DC, USA
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

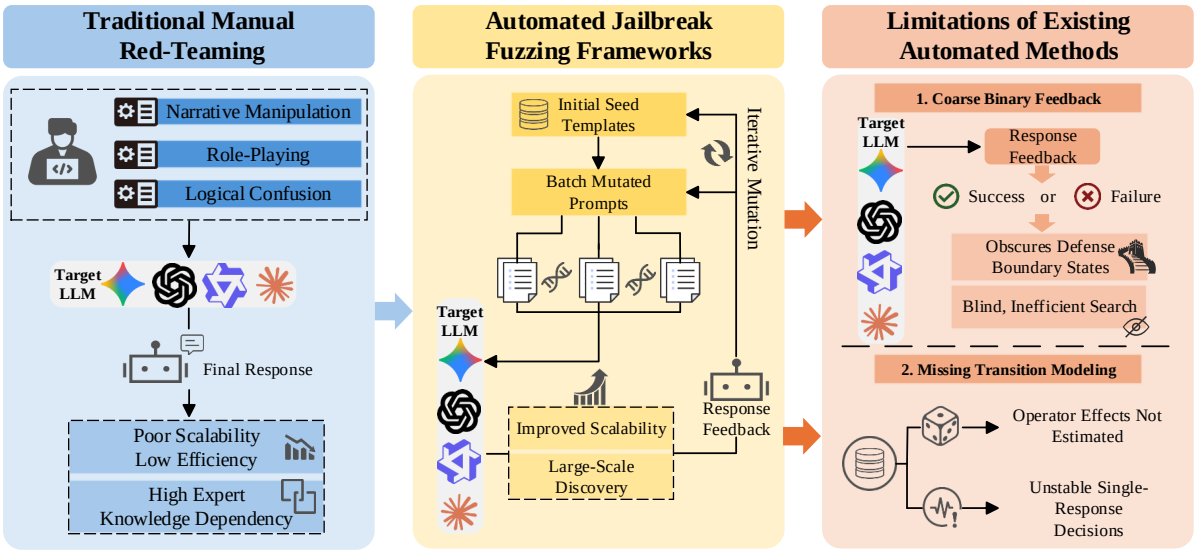

Figure 1: Evolution and limitations of jailbreak testing strategies. (Left) Traditional manual red-teaming relies on expert-designed prompts and suffers from limited scalability. (Middle) Automated jailbreak fuzzing improves scalability by mutating seed templates and using target responses as feedback. (Right) Existing methods still face two limitations: binary feedback causes state aliasing near the defense boundary, and mutation/search decisions lack defense-state transition modeling, leading to inefficient and unstable jailbreak discovery.

for later mutation. Second, mutation and search decisions are often driven by single outcomes rather than stable defense-state changes. Existing fuzzers rarely estimate how a mutation operator changes the target model's defense state under the current prompt. Consequently, the search may select ineffective mutations, overlook promising intermediate templates, and report jailbreaks that cannot be reproduced reliably. These failures reduce query efficiency and make the final results less trustworthy.

To address these limitations, we propose DeST-Fuzzing, a defense-state transition optimization framework for automated jailbreak fuzzing. Instead of treating target feedback as a binary outcome, DeST-Fuzzing estimates four ordered defense states for each response and converts them into a boundary potential that measures progress toward policy bypass. It then models each mutation operator as a state-transition action. During search, the framework estimates the potential gain of each mutation operator under the current defense state and uses this estimate to guide mutation selection. To preserve intermediate progress, DeST-Fuzzing organizes prompt templates in a non-linear search tree with dynamic intermediate expansion. To avoid unstable results, it measures response uncertainty under repeated target-model calls and uses uncertainty-gated value propagation to suppress high-uncertainty branches. This design improves feedback granularity, mutation effectiveness, and reproducibility within a unified search process.

The contributions can be summarized as follows:

- We formulate automated jailbreak fuzzing as a defense-state transition optimization problem for black-box LLM red teaming, where target responses define ordered defense states and search aims to discover stable transition paths from refusal to acceptance.
- We design a transition-aware mutation policy that models mutation operators as state-transition actions. By estimating state-dependent potential gains, the policy selects operators suited to the current defense state, reducing ineffective transformations during fuzzing.
- We develop a stability-aware non-linear tree search with dynamic intermediate expansion and uncertainty-gated value propagation. The search preserves promising intermediate templates, suppresses unstable branches under response uncertainty, and prioritizes reproducible jailbreak paths.

2 Related Work

2.1 Safety Alignment of LLMs

Safety alignment serves as the primary defense against adversarial exploitation. While models trained on massive datasets demonstrate advanced capabilities [1, 5, 11], they also present risks of generating harmful content. To mitigate this, safety alignment has become a standard phase in model development. The current approach typically relies on Reinforcement Learning from Human Feedback (RLHF) [9, 28], preceded by Supervised Fine-Tuning (SFT) on curated safety datasets [14]. During RLHF, a reward model trained on human preferences penalizes outputs that violate safety policies, guiding the LLMs toward helpful and harmless behaviors [18]. Beyond human feedback, scalable oversight techniques such as Constitutional AI [3] utilize AI-generated critiques to enforce ethical principles. Recent methods, such as Self-Correction [37], further encourage models to refine their own outputs by detecting potential violations. These processes establish a complex defense boundary within the model's latent space. However, this boundary

is often not well-defined. Studies on adversarial robustness [38] indicate that while these mechanisms filter standard malicious queries, they often fail to generalize to out-of-distribution adversarial prompts. Understanding the instability of this alignment boundary is essential, as our work focuses on navigating its complex topology.

2.2 Adversarial Jailbreaking of LLMs

Jailbreak attacks are a specialized class of adversarial threats designed to circumvent safety alignment. Initial research focuses on white-box attacks, where adversaries have full access to the model's parameters. Works such as Greedy Coordinate Gradient [44, 45] demonstrate that optimizing adversarial suffixes via gradient-based search can force aligned models to generate prohibited content. Similarly, ARCA [16] leverages discrete optimization to compute adversarial tokens. However, the practicality of white-box methods is constrained in real-world scenarios, as many state-of-the-art models are deployed as black-box APIs. Consequently, research has shifted toward black-box attacks. Early efforts rely on manual prompt engineering, such as the *Do Anything Now* (DAN) prompt [33], which uses complex role-playing to bypass safety mechanisms. Other manual strategies exploit cognitive overload [40] or translation-based obfuscation [20]. While effective in certain cases, these human-crafted templates have limited scalability and require substantial domain expertise. This has led to the development of automated algorithmic solutions.

2.3 Automated Red Teaming Frameworks

To address the limitations of manual testing, automated red teaming has become a major research direction [6, 31, 41]. These frameworks often treat jailbreaking as a fuzzing problem [6, 25, 41], where seed templates are mutated and target-model responses are used to guide subsequent search. Approaches such as LLM-Fuzzer [41] and Arondight [22] adopt a mutation-based paradigm in which an attacker LLM rewrites seed templates to generate adversarial prompts. Similarly, PAIR [7] uses an attacker agent to refine prompts based on target responses. Genetic and optimization-based methods have also been explored. For instance, AutoDAN [21] utilizes hierarchical search to evolve stealthy prompts, while MasterKey [10] fine-tunes attacker models on successful examples. Despite these advances, existing frameworks usually use target responses as coarse terminal feedback. Binary success/failure labels cause state aliasing near the defense boundary, because complete refusals and partial acceptances are both treated as failures. Moreover, current mutation and search strategies rarely estimate the defense-state change produced by each mutation operator. As a result, they provide limited guidance for operator selection and may rely on unstable single-response outcomes, reducing query efficiency and reproducibility across attack attempts.

3 Methodology

3.1 Problem Formulation

We consider automated jailbreak fuzzing in a black-box red-teaming setting. The target large language model is denoted by M. The fuzzer submits textual prompts through an API and observes generated responses. Internal parameters, gradients, logits, hidden states, training data, and safety filter rules are unavailable. The available feedback consists of submitted prompts, returned responses, and defense states estimated from these responses.

Let Q denote a malicious query used for safety evaluation, and let T denote a prompt template. The submitted prompt is constructed as $P = T \oplus Q$, where $\oplus$ denotes textual concatenation under a fixed formatting rule. For a submitted prompt P, the target model returns a response $R \sim M(P)$. Repeated calls with the same P may yield different defense states because LLM APIs use stochastic decoding and may apply dynamic safety controls.

Let $\mathbb{T}$ denote the template space, and let $\mathcal{A}$ denote the set of mutation operators. A mutation operator $a \in \mathcal{A}$ maps one template to another template, $a : \mathbb{T} \to \mathbb{T}$. Operator descriptions and implementation constraints are provided in the appendix. Starting from an initial template T_0, DeST-Fuzzing constructs a mutation trajectory

$$\tau = (T_0, a_0, T_1, a_1, \dots, a_{H-1}, T_H), \quad T_{t+1} = a_t(T_t), \quad a_t \in \mathcal{A}. \tag{1}$$

Here, H is the maximum search depth. At step t, the submitted prompt is $P_t = T_t \oplus Q$.

The search is performed under a query budget. Let B denote the total number of target-model calls, and let Ω_B denote the set of trajectories whose calls to the target model do not exceed B. Calls used for state estimation and stability estimation are counted in B. Let $\Phi(T, Q) \in [0, 1]$ denote the boundary potential, and let $\widetilde{U}(T, Q) \in [0, 1]$ denote the normalized response-state uncertainty. These two quantities are defined in Section 3.2. The optimization objective is

$$\begin{aligned} \tau^\star &= \arg\max_{\tau \in \Omega_B} \sum_{t=0}^{H-1} \left[\Delta\Phi_t - \lambda \widetilde{U}(T_{t+1}, Q) \right], \\ \Delta\Phi_t &= \Phi(T_{t+1}, Q) - \Phi(T_t, Q), \end{aligned} \tag{2}$$

where $\lambda \geq 0$ controls the penalty assigned to unstable response-state distributions. Throughout the method, $\Delta\Phi_t$ serves as the quantitative signal for boundary progress. It measures the change in boundary potential caused by an evaluated mutation and is later used as the main reward signal for transition utility estimation and tree value update. The final output is selected from all evaluated templates by the stable ranking rule in Section 3.5.

3.2 State Estimation and Boundary Potential

Each evaluated template must be converted into state observations before it can be used by the transition model and the search tree. DeST-Fuzzing uses the four defense states in Table 1. The labels form an ordinal scale. A larger label indicates a higher degree of policy bypass reflected in the target-model response.

For a template T and a query Q, DeST-Fuzzing constructs $P = T \oplus Q$ and queries the target model K times. The repeated calls estimate the variability of the target response under the same submitted prompt. Each returned response is mapped to a defense state by the defense state estimator E:

$$R_i \sim M(\cdot \mid P), \quad P = T \oplus Q, \quad s_i = E(R_i), \quad i = 1, \dots, K. \tag{3}$$

R_i is the i-th response and $s_i \in S$ is its estimated defense state. The estimator is implemented as a discriminative response classifier trained on annotated target-model responses. It provides the state observations required by transition estimation and tree search.

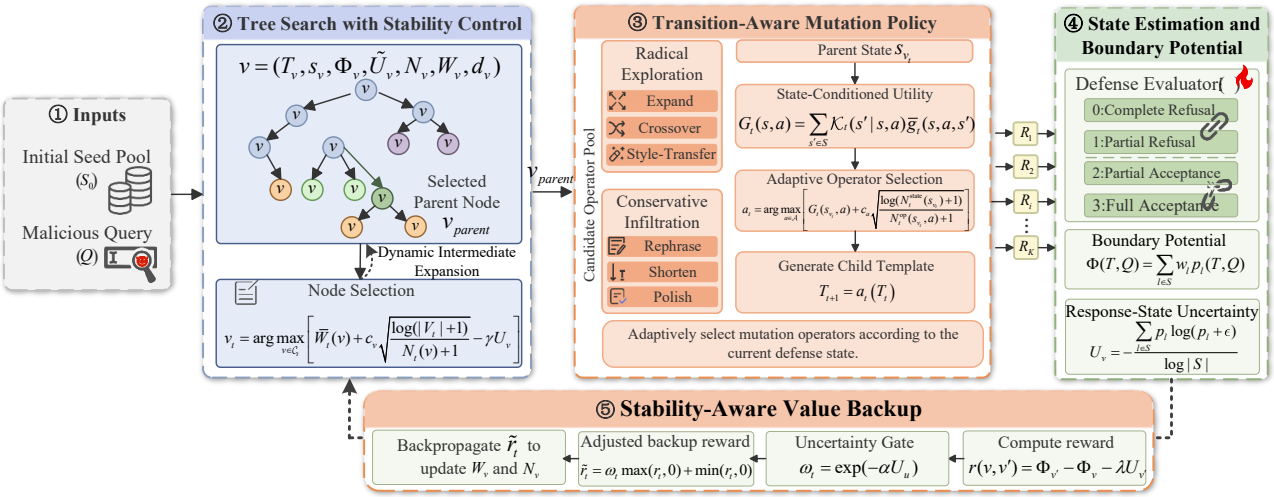

Figure 2: Overview of DeST-Fuzzing. The framework starts from seed templates and a malicious query, selects expandable nodes through stability controlled tree search, estimates state conditioned mutation utilities to generate child templates, and evaluates responses with a defense state estimator. Boundary potential and state uncertainty are then used to compute transition rewards, update node values and operator utilities, and iteratively guide the search toward stable high potential templates.

Table 1: Defense states used in DeST-Fuzzing.

Label	State	Meaning
0	Complete Refusal	The target model rejects the request and provides no task relevant unsafe content.
1	Partial Refusal	The target model recognizes the request context while still refusing the unsafe intent.
2	Partial Acceptance	The response contains task relevant content, while the content remains incomplete, vague, or constrained by safety behavior.
3	Full Acceptance	The response provides detailed content that violates the safety policy used in the evaluation.

The K state observations are summarized as an empirical response-state distribution:

$$p_l(T,Q) = \frac{1}{K}\sum_{i=1}^{K} \mathbf{1}[s_i = l], \quad s(T,Q) = \arg\max_{l\in S} p_l(T,Q). \tag{4}$$

The value $p_l(T,Q)$ estimates how often template T induces state l for query Q. The compact label $s(T,Q)$ is the most frequent state and is used as the categorical index for transition statistics. Ties are resolved by selecting the smaller label, which gives a conservative state estimate when the response distribution is ambiguous.

The scalar progress signal is computed from the full distribution $p_l(T,Q)$. Since the state labels are ordinal categories, DeST-Fuzzing uses monotone calibration weights and avoids assuming equal semantic distance between adjacent labels. Let w_0, w_1, w_2, w_3 satisfy $0 = w_0 < w_1 < w_2 < w_3 = 1$. The boundary potential is

$$\Phi(T,Q) = \sum_{l\in S} w_l p_l(T,Q). \tag{5}$$

A larger $\Phi(T,Q)$ means that repeated responses place more probability mass on higher bypass states. The difference between two boundary potentials gives the $\Delta\Phi_t$ term in Equation 2. Thus, the state label $s(T,Q)$ groups observations for transition counting, and the boundary potential supplies the scalar value used to score mutation progress.

The same response-state distribution also measures stability. DeST-Fuzzing defines the normalized response-state uncertainty as

$$\widetilde{U}(T,Q) = \frac{-\sum_{l\in S} p_l(T,Q)\log(p_l(T,Q)+\epsilon)}{\log|S|}, \tag{6}$$

where ϵ is a small numerical constant. This entropy is close to zero when repeated calls yield the same state and increases when probability mass spreads across multiple states. The uncertainty term is used to reduce the credit assigned to unstable positive outcomes in later search stages.

3.3 Defense State Transition Model

The transition model connects state estimation to mutation selection. It records how each mutation operator changes the defense state observed from a template. The defense state transition graph is

$$\mathcal{G} = (S, \mathcal{A}, \mathcal{K}), \quad S = \{0,1,2,3\}. \tag{7}$$

Here, S is the defense state space, $\mathcal{A}$ is the mutation operator set, and $\mathcal{K}$ is the transition kernel induced by target-model randomness and mutation randomness.

For a current state s and a selected mutation operator a, the transition kernel is

$$\mathcal{K}(s' \mid s,a) = \mathbb{P}(s_{t+1} = s' \mid s_t = s, a_t = a), \quad s,s' \in S. \tag{8}$$

For each fixed pair (s, a), the distribution $\mathcal{K}(\cdot \mid s, a)$ describes the probabilities that operator moves a template from the current defense state to each possible next state. This distribution allows the fuzzer to estimate whether an operator tends to preserve, regress, or improve the defense state under the current search context.

The transition pair (s_t, s_{t+1}) and the potential increment $\Delta\Phi_t$ both describe the effect of the same evaluated mutation. The former records the change at the level of discrete defense states, indicating whether the mutation leads to state preservation, progress, or regression. The latter measures the change at the level of boundary potential, quantifying how much progress or regression is produced by the mutation. DeST-Fuzzing uses (s_t, s_{t+1}) to update the empirical transition kernel of the selected operator, and uses $\Delta\Phi_t$ as the scalar reward for operator ranking and tree value update.

The true kernel is unknown before search, so DeST-Fuzzing estimates it online. Let $C_t(s, a, s')$ be the number of observed transitions from s to s' under operator a before iteration t, and let $N_t^{\mathrm{op}}(s, a) = \sum_{\bar{s} \in S} C_t(s, a, \bar{s})$. With smoothing constant $\beta > 0$, the empirical transition kernel is

$$\widehat{\mathcal{K}}_t(s' \mid s, a) = \frac{C_t(s, a, s') + \beta}{N_t^{\mathrm{op}}(s, a) + |S|\beta}. \tag{9}$$

The numerator counts the specific transition type (s, a, s'). The denominator normalizes over all possible next states under the same (s, a) pair. The smoothing term assigns a small probability to unobserved next states during early search. After observing (s_t, a_t, s_{t+1}), DeST-Fuzzing increments $C_t(s_t, a_t, s_{t+1})$ by one and recomputes the empirical kernel from the updated counts.

The kernel alone gives the probability of next states. Mutation selection also needs the value of reaching those states. DeST-Fuzzing therefore records the average reward associated with each transition type. Let $I_i(s, a, s') = \mathbf{1}[s_i = s, a_i = a, s_{i+1} = s']$ and $\widetilde{U}_{i+1} = \widetilde{U}(T_{i+1}, Q)$. The average transition reward is

$$\bar{g}_t(s, a, s') = \frac{\sum_{i<t} I_i(s, a, s') \left(\Delta\Phi_i - \lambda \widetilde{U}_{i+1}\right)}{C_t(s, a, s') + \epsilon}, \tag{10}$$

where $\epsilon > 0$ is a small numerical constant. The numerator accumulates the boundary-potential gains observed for the same transition type and subtracts the uncertainty penalty of the resulting template. Thus, $\bar{g}_t$ estimates the reward obtained when operator a moves state s to state s'.

The state-conditioned operator utility combines the probability of each next state with the reward observed for reaching that state:

$$G_t(s, a) = \sum_{s' \in S} \widehat{\mathcal{K}}_t(s' \mid s, a) \bar{g}_t(s, a, s'). \tag{11}$$

This expectation gives the transition kernel a direct role in operator selection. An operator receives a larger utility in state s when its observed next-state distribution assigns more probability to transitions with larger potential gains and lower response-state uncertainty. In this way, the transition model converts historical state movements into a state-conditioned estimate of operator effectiveness.

3.4 Stability-Controlled Tree Search

DeST-Fuzzing organizes evaluated templates in a non-linear template tree. The tree is denoted by $\mathcal{T}_{\mathrm{tree}} = (V, \mathcal{E})$. Each node $v \in V$ stores

$$v = \left(T_v, s_v, \Phi_v, \widetilde{U}_v, N_v, W_v, d_v\right). \tag{12}$$

Here, T_v is the template, $s_v = s(T_v, Q)$ is the template-level defense state, $\Phi_v = \Phi(T_v, Q)$ is the boundary potential, $\widetilde{U}_v = \widetilde{U}(T_v, Q)$ is the normalized response-state uncertainty, N_v is the visit count, W_v is the accumulated value, and d_v is the tree depth. An edge $(v, u) \in \mathcal{E}$ records that child node u is generated by applying one mutation operator to T_v. Nodes with $s_v = 3$ are retained in the candidate output set and excluded from regular expansion.

The tree-specific statistics $W_t(v)$ and $N_t(v)$ summarize previous value backups before iteration t. Each completed expansion j creates a child node u_j and produces a scalar backup reward $\tilde{r}_j$, whose computation is described in Section 3.5. The reward is passed to u_j and to all ancestor nodes of u_j. Therefore, before iteration t, the cumulative tree value and backup count of node v are

$$\begin{aligned} W_t(v) &= \sum_{j<t} \mathbf{1}\left[v \in \mathsf{Anc}(u_j) \cup \{u_j\}\right] \tilde{r}_j \\ N_t(v) &= \sum_{j<t} \mathbf{1}\left[v \in \mathsf{Anc}(u_j) \cup \{u_j\}\right] \end{aligned} \tag{13}$$

Both quantities are initialized as zero when a node is inserted. Thus, $W_t(v)$ is the accumulated stable progress obtained from node v and its evaluated descendants, and $N_t(v)$ counts how many backup updates have reached v. The mean value used for parent selection is

$$\bar{W}_t(v) = \frac{W_t(v)}{N_t(v) + 1}. \tag{14}$$

The added one prevents division by zero and keeps newly inserted nodes well defined before they receive backup updates.

Dynamic intermediate expansion determines which nodes can serve as the parent of the next mutation. Leaf nodes are natural candidates because they have not been expanded. Some internal nodes should also remain expandable. This occurs when an internal node has enough search evidence, while its existing children have not achieved a clear value improvement over the parent. In that case, the parent template may still be useful, and a different mutation can create an alternative branch from the same intermediate template.

Let $\mathsf{Ch}(v)$ be the child set of v. An internal node is marked for dynamic intermediate expansion when

$$\chi_t(v) = \mathbf{1}\begin{bmatrix} |\mathsf{Ch}(v)| > 0, \quad N_t(v) \geq n_{\min}, \\ \max_{u \in \mathsf{Ch}(v)} \bar{W}_t(u) \leq \bar{W}_t(v) + \delta_{\mathrm{die}} \end{bmatrix}. \tag{15}$$

The threshold $n_{\min}$ requires minimal evidence before an internal node is reconsidered. The tolerance δ_{die} specifies the improvement margin required from the best child. If the best child remains within this margin, DeST-Fuzzing permits another expansion from the parent. This rule preserves intermediate templates whose existing descendants have not fully used their search value.

The expandable candidate set is

$$\mathcal{C}_t = \{v \in V_t \,:\, s_v < 3,\ d_v < H,\ |\mathsf{Ch}(v)| = 0 \ \vee\ \chi_t(v) = 1\}. \tag{16}$$

This set contains ordinary leaf nodes and qualified intermediate nodes. The condition $s_v < 3$ stops regular expansion of full-acceptance nodes, and the condition $d_v < H$ enforces the depth budget. The

last condition gives dynamic intermediate expansion its role in the search loop by allowing selected internal nodes to branch again.

DeST-Fuzzing selects the parent node from $\mathcal{C}_t$ using a stability-controlled score:

$$v_t = \arg\max_{v \in \mathcal{C}_t} \left[\bar{W}_t(v) + c_v \sqrt{\frac{\log(|V_t| + 1)}{N_t(v) + 1}} - \gamma \widetilde{U}_v \right]. \tag{17}$$

The first term favors nodes whose previous descendants produced larger stable progress. The second term allocates trials to less-visited nodes and prevents the search from concentrating on a small set of early high-value nodes. The third term lowers the priority of nodes whose repeated target-model calls produce unstable response-state distributions. The constants $c_v \geq 0$ and $\gamma \geq 0$ are fixed during each experimental run. The selected node v_t is passed to the transition-aware expansion step.

3.5 Transition-Aware Expansion and Value Update

Given the selected parent node v_t, DeST-Fuzzing chooses a mutation operator according to the defense state stored in the parent node and the online transition statistics. The operator set $\mathcal{A}$ contains radical exploration operators and conservative refinement operators. These categories describe the editing scale of the operators. All operators remain available for each expandable state, and the final choice is determined by state-conditioned utility and an exploration bonus.

Let s_{v_t} be the defense state stored in parent node v_t. The quantity $N_t^{\text{op}}(s, a)$ defined in Section 3.3 counts how many times operator a has been selected under state s. For operator-level exploration, DeST-Fuzzing also uses the total number of operator selections made under state s:

$$N_t^{\text{state}}(s) = \sum_{a \in \mathcal{A}} N_t^{\text{op}}(s, a). \tag{18}$$

The selected operator is

$$a_t = \arg\max_{a \in \mathcal{A}} \left[G_t(s_{v_t}, a) + c_a \sqrt{\frac{\log(N_t^{\text{state}}(s_{v_t}) + 1)}{N_t^{\text{op}}(s_{v_t}, a) + 1}} \right]. \tag{19}$$

The utility term $G_t(s_{v_t}, a)$ exploits the empirical transition kernel from Section 3.3. The exploration term assigns trials to operators with fewer observations under the current state. The constant $c_a \geq 0$ controls the exploration strength.

After applying a_t, the child template is $T_u = a_t(T_{v_t})$. DeST-Fuzzing evaluates T_u by the state-estimation procedure in Section 3.2 and creates a child node u. The immediate transition reward is aligned with Equation 2:

$$r_t = (\Phi_u - \Phi_{v_t}) - \lambda \widetilde{U}_u. \tag{20}$$

The term $\Phi_u - \Phi_{v_t}$ is the boundary-potential increment of the evaluated mutation. The uncertainty penalty reduces the reward assigned to a child whose repeated responses are inconsistent.

A high reward from an unstable child can cause the tree to overvalue an unreliable branch. DeST-Fuzzing therefore applies an uncertainty gate before value backup. The gate is a scalar attenuation factor determined by the response-state uncertainty of the child node:

$$\omega_t = \exp(-\alpha \widetilde{U}_u), \quad \tilde{r}_t = \omega_t \max(r_t, 0) + \min(r_t, 0), \tag{21}$$

where $\alpha \geq 0$ controls the attenuation strength. When $\widetilde{U}_u$ is small, ω_t remains close to one, so positive transition reward is largely preserved. When $\widetilde{U}_u$ is large, ω_t becomes smaller and the positive reward passed to ancestor nodes is reduced. Negative rewards are kept unchanged through the $\min(r_t, 0)$ term, so regression evidence remains visible during value backup.

For every $z \in \text{Anc}(u) \cup \{u\}$, DeST-Fuzzing updates the tree statistics by

$$N_{t+1}(z) = N_t(z) + 1, \quad W_{t+1}(z) = W_t(z) + \tilde{r}_t. \tag{22}$$

This recursive update is equivalent to the cumulative definition in Equation 13. The same evaluated mutation also updates the transition statistics. The observed triple (s_{v_t}, a_t, s_u) increments the corresponding transition count, the empirical kernel in Equation 9 is recomputed from the updated counts, and the utility in Equation 11 is refreshed. Thus, each target-model evaluation contributes to parent selection through tree value and to operator selection through transition utility.

After the query budget is exhausted, DeST-Fuzzing selects the final template from evaluated nodes. If at least one node has $s_v = 3$, the output candidate set contains those nodes. Otherwise, all evaluated nodes are considered. The returned node maximizes $\Phi_v - \lambda_{\text{out}} \widetilde{U}_v$ within the output candidate set, and its template is returned as $T^\star$. This ranking favors templates with large boundary potential under repeated evaluation and a smaller response-state uncertainty penalty.

3.6 Algorithm Summary

Algorithm 1 summarizes DeST-Fuzzing. The algorithm initializes a non-linear template tree from seed templates, estimates response-state distributions by repeated target-model calls, and then alternates among parent selection, transition-aware operator selection, child evaluation, uncertainty-gated value update, and transition-statistic update. Each evaluated mutation records a defense-state transition and updates both tree statistics and empirical transition utilities.

4 Experiments

4.1 Experimental Setup

We evaluate DeST-Fuzzing in a black-box red-team auditing setting. Each method receives the same initial seed pool, the same safety-evaluation query set, and the same accounting rule for target-model calls. Internal parameters, gradients, logits, hidden states, training data, and safety-filter rules of target models are unavailable. All calls used for defense-state estimation, response-state uncertainty estimation, and repeated verification are counted in the query budget. We report aggregate statistics and omit generated templates, mutation chains, and target-model response excerpts.

Seed templates and evaluation queries. The automated search process is initialized using a repository of human-authored jailbreak templates derived from [23]. We select 80 templates with distinct structural patterns and high mutation potential to form the seed pool. These templates serve as the root nodes for the search

Algorithm 1 DeST-Fuzzing

Require: Target LLM M, query Q, seed templates $\mathcal{T}_0$, defense state estimator E, mutation operators $\mathcal{A}$, sampling number K, query budget B, maximum depth H
Ensure: Selected template $T^\star$

Initialize $\mathcal{T}_{\text{tree}} = (V, \mathcal{E})$, transition counts $C(s, a, s')$, utilities $G(s, a)$, and query counter $q \leftarrow 0$
for each seed template $T \in \mathcal{T}_0$ **do**
  Query M with $P = T \oplus Q$ for K times and obtain $\{R_i\}_{i=1}^K$
  Estimate $\{s_i = E(R_i)\}_{i=1}^K$, then compute $s(T, Q)$, $\Phi(T, Q)$, and $\widetilde{U}(T, Q)$
  Insert the seed node into $\mathcal{T}_{\text{tree}}$ with $N = 0$, $W = 0$, and depth 0
  $q \leftarrow q + K$
end for
while $q + K \leq B$ **do**
  Construct $\mathcal{C}_t$ using Equation 16
  if $\mathcal{C}_t = \emptyset$ **then**
    break
  end if
  Select parent node v_t using Equation 17
  Select mutation operator a_t using Equation 19
  Generate child template $T_u = a_t(T_{v_t})$
  Query M with $P_u = T_u \oplus Q$ for K times and obtain $\{R_i\}_{i=1}^K$
  Estimate $\{s_i = E(R_i)\}_{i=1}^K$, then compute s_u, Φ_u, and $\widetilde{U}_u$
  Insert child node u under v_t and set its depth to $d_{v_t} + 1$
  Compute r_t and $\tilde{r}_t$ using Equations 20 and 21
  Update tree values for u and its ancestors using Equation 22
  Increment $C(s_{v_t}, a_t, s_u)$, recompute $\widehat{\mathcal{K}}(s' \mid s, a)$, and update $G(s, a)$
  $q \leftarrow q + K$
end while
Select $T^\star$ by maximizing $\Phi_v - \lambda_{\text{out}} \widetilde{U}_v$ over the output candidate set
return $T^\star$

tree and provide the initial semantic context for the search process. To construct attack payloads, we sample 100 malicious queries from two publicly available datasets [6, 42]. These queries cover various prohibited scenarios, such as illegal activities, discriminatory hate speech, and the generation of harmful materials. These datasets are utilized as they contain queries either designed by security experts or collected through crowdsourcing, ensuring that the evaluation reflects real-world red-teaming scenarios characterized by diverse and context-dependent inputs.

Target models and mutator model. GPT-3.5-Turbo is used as the mutator model for executing mutation-operator instructions. Structural-edit operators use temperature values from 0.8 to 1.0, and refinement operators use temperature 0.5. The target-model pool includes Claude 3.5 Sonnet [2], GPT-4 [1], GPT-3.5-Turbo, Llama-2-7B and Llama-2-13B [36], Llama 3.1 [11], and RealSafe-R1 [43].

Defense-state estimator. The defense-state estimator E is a RoBERTa-large classifier [24] trained on annotated target-model responses. We combine the 80 seed templates with the 100 evaluation queries and collect 8,000 target-model responses using GPT-3.5-Turbo. Four annotators label the responses according to the four-state taxonomy in Table 1. Ambiguous cases are resolved through consensus review. The annotated data are split into 80% training and 20% validation partitions. The classifier is fine-tuned for 15 epochs with batch size 80, maximum sequence length 512, learning rate 1×10^{-5}, Adam optimization [17], linear learning-rate decay, 10% warm-up, and mixed precision [27].

Baselines. We compare DeST-Fuzzing with four automated jailbreak testing baselines. LLM-Fuzzer is used as the main mutation-based fuzzing baseline because it follows a seed-driven iterative search paradigm close to our setting. ArtPrompt represents prompt-transformation methods that rewrite harmful requests into alternative surface forms. LLMjailbreak is included as a strong template-based baseline for evaluating manually designed jailbreak patterns. SimBAja serves as a black-box optimization baseline based on iterative candidate refinement. All methods are evaluated with the same query set, target-model pool, and budget accounting whenever their implementations allow direct control.

Default budgets and verification. Unless stated otherwise, the hard-subset experiments use a per-query target-model budget of $B = 300$ calls, maximum tree depth $H = 6$, and $K = 3$ repeated calls for state and uncertainty estimation. Repeated verification uses $K_{\text{ver}} = 5$ calls for each selected template-query pair. The default search parameters are $c_v = 1.2$, $c_a = 0.7$, $\lambda = 0.3$, $\alpha = 2.0$, $n_{\min} = 2$, and $\delta_{\text{die}} = 0.02$.

4.2 Experimental Design and Metrics

The experiments are organized around six research questions.

RQ1: How much vulnerability can be exposed by static seed templates before automated search? This question measures the baseline effectiveness of the initial seed pool and identifies hard queries that cannot be solved by direct template reuse.

RQ2: Can DeST-Fuzzing improve discovery on hard queries under different seed-quality settings? This question evaluates whether the proposed search process can make progress when the initial seed pool contains valid, invalid, or mixed templates.

RQ3: Does DeST-Fuzzing maintain effective and stable performance across different target models? This question evaluates cross-model query coverage, query efficiency, and repeated-verification stability.

RQ4: Is the defense-state estimator reliable enough to support transition-based search? This question validates whether the response classifier can provide usable defense-state labels for boundary potential, uncertainty estimation, and transition rewards.

RQ5: Which components and transition behaviors explain the performance of DeST-Fuzzing? This question examines the contribution of each algorithmic component and analyzes whether mutation operators exhibit state-dependent effects.

RQ6: How sensitive is DeST-Fuzzing to query budget, repeated state estimation, and uncertainty-control parameters? This question studies whether the method depends on a narrow configuration or remains stable under different evaluation costs and hyperparameter settings.

Table 2: Static seed effectiveness on three target models. Avg. Succ. denotes the average number of successful seed templates per query. Invalid denotes the number of seed templates that produce zero successful cases over the query set.

Model	JQN	Top-1	Top-5	Avg. Succ.	Invalid
Llama 3.1	100/100	100%	100%	62.69	3
RealSafe-R1	100/100	66%	100%	16.21	2
Claude 3.5 Sonnet	52/100	52%	52%	0.52	79

Let $\mathcal{Q}$ denote the evaluation-query set and let $\mathcal{T}_{\text{gen}}$ denote the templates generated by a method. Let $I_{\text{succ}}(T,q) = 1[s(T,q) = 3]$, where state 3 denotes Full Acceptance under the evaluation taxonomy. We report Jailbroken Query Number, Top-1 ASR, Top-5 ASR, and Average Queries to Success:

$$\text{JQN} = \sum_{q \in \mathcal{Q}} \mathbf{1}\left[\exists T \in \mathcal{T}_{\text{gen}}, I_{\text{succ}}(T,q) = 1\right], \tag{23}$$

$$\text{ASR}(T) = \frac{1}{|\mathcal{Q}|} \sum_{q \in \mathcal{Q}} I_{\text{succ}}(T,q), \quad \text{Top1} = \max_{T \in \mathcal{T}_{\text{gen}}} \text{ASR}(T), \tag{24}$$

$$\text{Top5} = \frac{1}{|\mathcal{Q}|} \sum_{q \in \mathcal{Q}} \mathbf{1}\left[\exists T \in \mathcal{T}_{\text{top5}}, I_{\text{succ}}(T,q) = 1\right], \quad \text{AvgQ} = \frac{N_{\text{total}}}{N_{\text{success}}}. \tag{25}$$

Here, $\mathcal{T}_{\text{top5}}$ contains the five templates with the largest ASR, N_{total} is the number of target-model calls, and N_{success} is the number of successful events. To evaluate stability, a template-query pair is stably successful when the repeated-verification success frequency reaches $\rho = 0.5$:

$$I_{\text{stable}}(T,q) = \mathbf{1}\left[\frac{1}{K_{\text{ver}}} \sum_{i=1}^{K_{\text{ver}}} I_{\text{succ}}^{(i)}(T,q) \geq \rho\right]. \tag{26}$$

Stable ASR replaces I_{succ} with I_{stable} in the ASR computation. Reproduction Rate is the fraction of initially successful template-query pairs that remain successful under repeated verification. For transition analysis, we report mean boundary-potential gain $\Delta\Phi$, Positive Transition Rate, Regression Rate, State-3 Transition Ratio, and mean response-state uncertainty $\widetilde{U}$.

4.3 Baseline Effectiveness of Initial Seed Templates

To answer RQ1, we first evaluate the 80 initial seed templates without applying mutation operators. Each seed template is paired with each of the 100 evaluation queries. This static configuration measures how much vulnerability can be exposed by the seed library alone and identifies the hard subset that requires automated search.

Table 2 shows different sensitivity levels to the same static seed library. Llama 3.1 and RealSafe-R1 reach JQN values of 100/100, indicating that the initial templates already cover all evaluation queries on these two targets. However, their template-level success counts differ substantially. Llama 3.1 has 62.69 successful templates per query on average, while RealSafe-R1 has 16.21, suggesting that RealSafe-R1 is less broadly affected by redundant seed templates even when query-level coverage is complete. Claude 3.5 Sonnet is more resistant to the static seed pool. The seed templates cover only 52 out of 100 evaluation queries, and 79 of the 80 templates produce zero successful cases on this target. The remaining 48 queries form the hard subset used in later experiments, where automated mutation and transition-aware search are required.

4.4 Automated Search on the Hard Subset

To answer RQ2, we evaluate DeST-Fuzzing and LLM-Fuzzer on the 48 Claude 3.5 Sonnet queries unsolved by the static seed library. Both methods use the same query budget, mutator model, seed pool, and target access constraints. We test three initialization settings: All uses the complete 80-template pool, Valid uses the single successful static seed, and Invalid uses the 79 unsuccessful seeds. This design evaluates performance under mixed, strong, and weak seed conditions.

Table 3 shows that DeST-Fuzzing obtains larger hard-subset coverage than LLM-Fuzzer in all three initialization settings. The All setting indicates that DeST-Fuzzing can extract useful search signals from a mixed seed pool in which most templates are weak for Claude 3.5 Sonnet. This result is important because realistic red-team audits usually start from heterogeneous templates rather than a known effective seed. The Valid setting shows that both methods benefit from a strong initial entry point, while DeST-Fuzzing converts this advantage into full hard-subset coverage with fewer queries. The Invalid setting is more diagnostic of boundary navigation. Although all initial seeds fail in the static experiment, DeST-Fuzzing still recovers 13 hard queries, suggesting that boundary potential and transition-aware operator selection can preserve intermediate progress before a full-success state appears.

4.5 Cross-Model Comparison

To answer RQ3, we evaluate whether DeST-Fuzzing maintains effective and stable performance across different target models. All compared methods are initialized with the full 80-template seed pool and evaluated on the same 100-query set. The baselines include ArtPrompt [15], LLMJailbreak [42], SimBAja [34], and LLM-Fuzzer [41]. We report JQN for cross-method comparison, and further report Top-1 ASR, Top-5 ASR, AvgQ, Stable ASR, and mean response-state uncertainty $\widetilde{U}$ for DeST-Fuzzing.

Figure 3 shows that DeST-Fuzzing obtains the largest or tied-largest JQN on all five target models. It reaches 95 on Claude 3.5 Sonnet, 100 on GPT-3.5-Turbo, 100 on GPT-4, 100 on Llama-2-7B, and 95 on Llama-2-13B. The gains are more evident on resistant targets. On Claude 3.5 Sonnet, DeST-Fuzzing reaches 95 JQN, while ArtPrompt, LLMjailbreak, SimBAja, and LLM-Fuzzer reach 1, 78, 81, and 83, respectively. On GPT-4, DeST-Fuzzing reaches full coverage, while ArtPrompt, SimBAja, and LLM-Fuzzer reach 32, 85, and 59. These results indicate that defense-state transition feedback improves cross-model query coverage.

Figure 4 further analyzes efficiency and stability. Top-5 ASR remains high on all targets, ranging from 96% on Claude 3.5 Sonnet to 100% on GPT-3.5-Turbo, GPT-4, and Llama-2-7B. Claude 3.5 Sonnet requires the largest search cost, with AvgQ of 132.47, while GPT-3.5-Turbo and Llama-2-7B require 34.60 and 41.25. This agrees with the hard-subset results, where Claude 3.5 Sonnet is less sensitive to the initial seed library. For stability, Claude 3.5 Sonnet has the lowest Stable ASR at 82% and the highest mean uncertainty

Table 3: Comparison of automated jailbreaking performance on the Claude 3.5 Sonnet with 48 hard questions. "All" denotes using the full seed pool, "Valid" uses only the single effective seed, "Invalid" uses the 79 ineffective seeds, "Top-1" and "Top-5' denote the success rate of jailbreak attacks against all malicious questions for the top one and the top five templates.

Initial Seeds	JQN	Avg. Queries	Top-1 ASR	Top-5 ASR
All	3/48	283.00	20.00%	28.00%
Valid	33/48	118.23	91.00%	96.00%
Invalid	1/48	299.90	1.00%	1.00%

(a) LLM-Fuzzer

Initial Seeds	JQN	Avg. Queries	Top-1 ASR	Top-5 ASR
All	32/48	132.47	87.00%	96.00%
Valid	48/48	40.00	93.00%	100.00%
Invalid	13/48	257.25	79.00%	97.00%

(b) Our DeST-Fuzzing

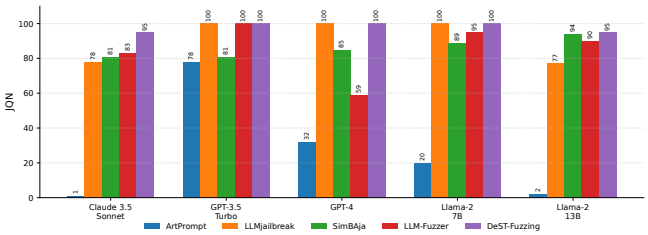

Figure 3: Cross-model comparison measured by JQN over 100 evaluation queries. Higher values indicate broader query coverage under the same evaluation set.

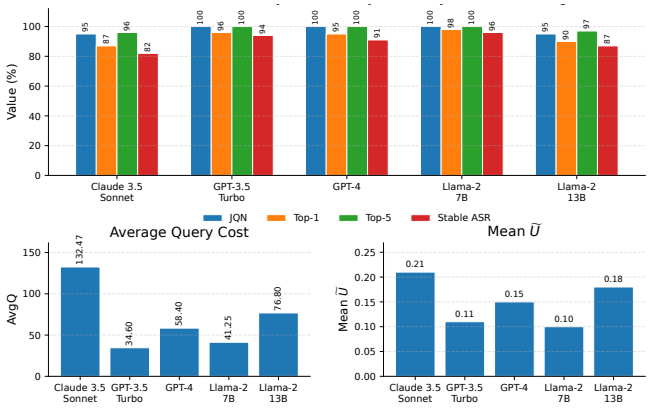

Figure 4: Cross-model efficiency and stability summary for DeST-Fuzzing across five target models.

at $\widetilde{U} = 0.21$, whereas Llama-2-7B has Stable ASR of 96% and the lowest uncertainty at $\widetilde{U} = 0.10$. Overall, DeST-Fuzzing improves cross-model coverage while keeping efficiency and stability measurable through repeated verification.

4.6 Defense-State Estimator Validation

To answer RQ4, we validate the defense-state estimator because DeST-Fuzzing relies on response-level state labels to compute boundary potential, response-state uncertainty, and transition rewards.

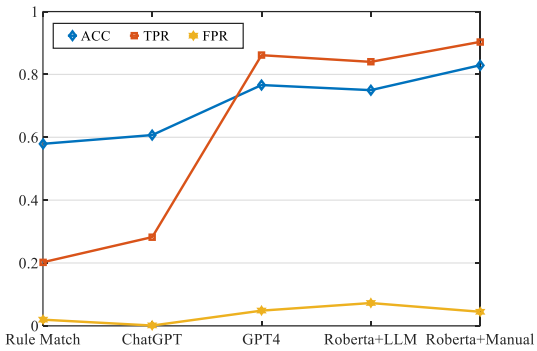

Figure 5: The Accuracy, TPR, and FPR of the five classification methods. GPT-3.5 and GPT-4 directly output classification results. Roberta+LLM indicates that the labels of all samples were determined by LLMs, while Roberta+Manual denotes that all samples were manually annotated.

The validation set consists of held-out target-model responses labeled according to the four defense states in Table 1. We compare five classification strategies: rule matching, direct classification by GPT-3.5, direct classification by GPT-4, RoBERTa trained with LLM-generated labels, and RoBERTa trained with manual labels. Accuracy measures the overall classification correctness. TPR measures the ability to identify positive high-risk states, and FPR measures the fraction of benign or lower-risk responses incorrectly classified as positive.

Figure 5 reports the validation results. Rule matching achieves limited accuracy and low TPR, indicating that keyword-level rules cannot capture many context-dependent response states. GPT-3.5 improves the overall accuracy slightly, but its TPR remains low in this validation setting. GPT-4 provides a stronger direct classifier, reaching higher accuracy and TPR than rule matching and GPT-3.5. However, prompted LLM classification requires additional model calls during fuzzing, which increases evaluation cost and may introduce response variance from the evaluator itself.

The two RoBERTa-based estimators provide more efficient discriminative classifiers for online search. RoBERTa trained with LLM-generated labels achieves competitive accuracy and TPR, showing that response classifiers can learn useful state patterns from automatically labeled data. RoBERTa trained with manual labels obtains the best overall result, with the highest accuracy and TPR

Table 4: Component ablation on the 48 hard Claude 3.5 Sonnet queries. JQN and Top-5 ASR measure effectiveness, AvgQ measures query efficiency, and Stable ASR and Reproduction Rate measure repeated-verification stability.

Method Variant	JQN	AvgQ	Top-5 ASR	Stable ASR	Reprod. Rate
DeST-Fuzzing	32/48	132.47	96.00%	82.00%	88.00%
w/o Boundary Potential	24/48	181.60	83.00%	70.00%	76.00%
w/o Transition-Aware Operator Selection	26/48	165.35	88.00%	74.00%	80.00%
w/o Dynamic Intermediate Expansion	27/48	158.42	89.00%	76.00%	81.00%
w/o Uncertainty-Gated Value Update	30/48	140.20	94.00%	71.00%	72.00%

Table 5: Transition-level diagnostics for component ablation. Mean $\Delta\Phi$ measures average boundary-potential gain. Positive Transition Rate denotes the fraction of mutations that move a template to a higher defense state, while Regression Rate denotes the fraction that move it to a lower state. State-3 Transition Ratio measures the fraction of evaluated mutations that reach Full Acceptance.

Method Variant	Mean $\Delta\Phi$	Pos. Trans.	Reg. Rate	State-3 Trans.	Mean $\widetilde{U}$
DeST-Fuzzing	0.118	46.20%	18.40%	7.80%	0.21
w/o Boundary Potential	0.071	34.10%	26.50%	4.20%	0.27
w/o Transition-Aware Operator Selection	0.083	38.30%	24.80%	5.10%	0.25
w/o Dynamic Intermediate Expansion	0.087	39.20%	25.60%	5.40%	0.23
w/o Uncertainty-Gated Value Update	0.112	45.10%	19.30%	7.50%	0.34

while keeping FPR low. This result supports using the manually supervised RoBERTa estimator as the default defense-state estimator in DeST-Fuzzing. It provides a stable and low-latency signal for boundary-potential estimation, transition-aware operator selection, and uncertainty-gated value update.

4.7 Component Ablation

To answer the component-level part of RQ5, we conduct ablation experiments on the 48 hard Claude 3.5 Sonnet queries using the full 80-template seed pool. All variants use the same query budget, mutator model, target access setting, and repeated-verification protocol. For w/o Boundary Potential, we replace boundary-potential reward with binary success feedback, where only state 3 receives positive reward. For w/o Transition-Aware Operator Selection, we remove $G_t(s, a)$ and use a state-agnostic UCB score for operator selection. For w/o Dynamic Intermediate Expansion, we restrict expansion to leaf nodes. For w/o Uncertainty-Gated Value Update, we set the uncertainty gate to one and backpropagate the raw transition reward.

Table 4 shows that each component contributes to final performance. Removing boundary potential causes the largest drop in JQN and Top-5 ASR, indicating that binary feedback provides weak guidance on hard queries. Removing transition-aware operator selection reduces coverage and increases AvgQ, showing the benefit of state-conditioned operator utility. Disabling dynamic intermediate expansion increases query cost, suggesting that reusing promising intermediate nodes improves search efficiency. Removing uncertainty-gated value update mainly lowers Stable ASR and Reproduction Rate, indicating that uncertainty control primarily improves reliability.

Table 5 further explains these trends. The full method obtains the largest mean $\Delta\Phi$, the highest positive transition rate, and the lowest regression rate. The binary-feedback variant weakens boundary-potential gains, while the state-agnostic operator variant reduces positive transitions. The variant without uncertainty-gated value update keeps similar transition progress but has higher mean $\widetilde{U}$. Overall, DeST-Fuzzing benefits from both fine-grained transition feedback and stability-aware search control.

4.8 Transition and Operator Diagnostics

To answer the transition-level part of RQ5, we further analyze the transition logs generated during the hard-subset search. Each evaluated mutation records its parent state, selected operator, child state, boundary-potential change, and response-state uncertainty.

Figure 6 reports operator-level statistics. Among all operators, Expand obtains the largest mean $\Delta\Phi$ and the highest positive transition rate, indicating that structural expansion is effective for moving templates out of refusal-dominated regions. Crossover and Style Transfer also produce relatively high positive transition rates, but their regression rates and mean $\widetilde{U}$ are larger than those of refinement operators. This suggests that larger edits can create useful state changes, while also introducing more unstable or regressive mutations. By contrast, Rephrase, Shorten, and Polish have smaller mean $\Delta\Phi$, but they generally show lower uncertainty. These operators are therefore less aggressive, but more stable when the current template already contains useful semantic structure.

Table 7 provides a clearer state-conditioned view. For parent states 0/1, radical exploration operators achieve a mean $\Delta\Phi$ of 0.119 and a positive transition rate of 45.80%, both higher than conservative infiltration operators. This confirms that larger structural edits are more useful when the target model remains in refusal

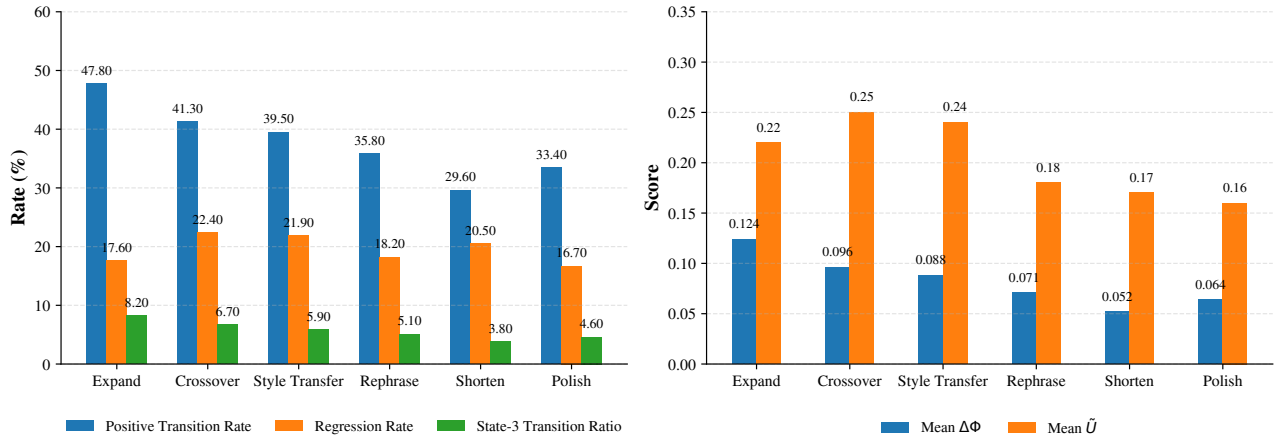

Figure 6: Operator-level transition diagnostics on the 48 hard Claude 3.5 Sonnet queries. Mean $\Delta\Phi$ measures average boundary-potential gain. Positive Transition Rate denotes the fraction of mutations that increase the defense state, while Regression Rate denotes the fraction that decrease it. State-3 Transition Ratio measures the fraction of mutations that reach Full Acceptance.

Table 6: Hyperparameter sensitivity on the 48 hard Claude 3.5 Sonnet queries. B controls the query budget, K controls the number of repeated calls for state estimation, λ controls the uncertainty penalty in transition reward, and α controls the attenuation strength of the uncertainty gate.

(a) Query budget B

B	JQN	AvgQ↓	Top-5	Stable
100	18/48	91.25	82.0%0	70.00%
200	27/48	114.80	91.00%	78.00%
300	32/48	132.47	96.00%	82.00%
400	34/48	158.20	97.00%	83.00%
500	35/48	189.60	97.00%	83.00%

(b) Sampling number K

K	JQN	AvgQ↓	Top-5	Stable
1	29/48	97.30	94.00%	68.00%
2	31/48	121.45	95.00%	77.00%
3	32/48	132.47	96.00%	82.00%
4	33/48	171.36	97.00%	85.00%
5	33/48	228.10	97.00%	85.00%

(c) Uncertainty penalty λ

λ	JQN	AvgQ↓	Top-5	Stable	$\widetilde{U}$
0.1	34/48	125.40	97.00%	72.00%	0.32
0.2	33/48	128.65	96.00%	78.00%	0.26
0.3	32/48	132.47	96.00%	82.00%	0.21
0.5	30/48	142.80	94.00%	83.00%	0.18
0.8	27/48	163.20	90.00%	82.00%	0.16

(d) Gate strength α

α	JQN	AvgQ↓	Top-5	Stable	$\widetilde{U}$
0.5	34/48	125.40	97.00%	73.00%	0.33
1.0	33/48	130.15	96.00%	79.00%	0.26
2.0	32/48	132.47	96.00%	82.00%	0.21
3.0	31/48	139.70	95.00%	83.00%	0.18
4.0	30/48	146.50	94.00%	82.00%	0.17

Table 7: State-conditioned transition diagnostics by operator group. Radical exploration (RE) operators include Expand, Crossover, and Style Transfer. Conservative infiltration (CI) include Rephrase, Shorten, and Polish.

Parent State	Operator	Mean $\Delta\Phi$	Pos. Trans.	Reg. Rate
State 0/1	RE	0.119	45.80%	18.90%
State 0/1	CI	0.061	30.70%	22.60%
State 2	RE	0.048	27.40%	29.80%
State 2	CI	0.093	42.10%	15.30%

states. For parent state 2, the pattern reverses. Conservative infiltration obtains a larger mean $\Delta\Phi$, a higher positive transition rate, and a lower regression rate than radical exploration. This indicates that near the defense boundary, local refinement is more effective than large structural rewriting because it preserves useful template components while reducing regression.

Overall, the diagnostics support the transition-aware mutation policy, i.e., operator selection should depend on the current defense state, and a fixed mutation schedule cannot capture these state-dependent effects.

4.9 Hyperparameter Sensitivity

To answer RQ6, we evaluate the sensitivity of DeST-Fuzzing to query budget, state-estimation sampling number, and uncertainty-control parameters on the 48 hard Claude 3.5 Sonnet queries. Each group changes one hyperparameter while keeping the others fixed. The default setting is $B = 300$, $K = 3$, $\lambda = 0.3$, and $\alpha = 2.0$.

Table 6 reports the sensitivity results. Increasing the query budget B improves JQN from 18/48 to 35/48, but the gain becomes smaller after $B = 300$ while AvgQ continues to increase. Top-5 ASR also improves from 82.00% to 96.00% when B increases from 100 to 300, and then remains nearly saturated. This indicates that a moderate budget already discovers most reachable hard cases, while larger budgets mainly provide marginal additional coverage at a higher query cost.

For the sampling number K, using a single response gives lower AvgQ but weaker Stable ASR, because response-state uncertainty cannot be estimated reliably from one target-model call. Increasing K improves Stable ASR from 68.00% to 82.00% when K increases from 1 to 3, showing that repeated state estimation helps suppress unstable observations. When K further increases to 4 or 5, the stability gain becomes limited, while AvgQ grows substantially. Therefore, $K = 3$ provides a practical balance between state-estimation reliability and query efficiency.

The uncertainty-control parameters mainly affect the trade-off between coverage and stability. Smaller λ or α values produce slightly higher JQN, but they also lead to lower Stable ASR and larger mean $\widetilde{U}$, indicating more unstable successful cases. Larger values reduce uncertainty, but overly strong penalties suppress exploration and decrease JQN. For example, increasing λ from 0.3 to 0.8 reduces mean $\widetilde{U}$ from 0.21 to 0.16, while JQN decreases from 32/48 to 27/48. A similar trend appears when α increases from 2.0 to 4.0. Overall, the default setting $B = 300$, $K = 3$, $\lambda = 0.3$, and $\alpha = 2.0$ provides a balanced configuration between hard-query coverage, query cost, and repeated-verification stability.

5 Discussion

5.1 Main Findings

The findings can be summarized from methodological and experimental perspectives. Methodologically, DeST-Fuzzing shows that automated jailbreak fuzzing benefits from modeling target responses as ordered defense states. Compared with binary feedback, defense-state representation preserves intermediate refusal and partial acceptance patterns. Boundary potential converts repeated response-state observations into a continuous search signal, and transition-aware mutation uses state-dependent evidence to select suitable operators. Together, these designs integrate feedback modeling, operator selection, tree expansion, and stability control into a unified search process.

Experimentally, the results are consistent with this design. The static seed experiment shows that Claude 3.5 Sonnet leaves 48 queries unresolved, forming a hard subset for automated search. On this subset, DeST-Fuzzing improves both coverage and query efficiency over LLM-Fuzzer under the same seed and budget setting. The cross-model comparison further shows broad query coverage across commercial and open-source targets. Component ablation and transition diagnostics indicate that boundary potential, transition-aware operator selection, and uncertainty control contribute to search effectiveness and stability. The operator analysis also shows state-dependent behavior, with structural operators more useful in refusal states and refinement operators more effective near state 2. Overall, the results support defense-state transition modeling as useful guidance for stable automated safety evaluation.

5.2 Limitations

This study has several limitations that should be considered when interpreting the results. First, the defense-state estimator introduces a learned measurement layer into the search loop. Although the estimator is validated on held-out responses, its behavior may change when target models use new refusal styles, mixed-policy responses, multilingual outputs, or domain-specific safety formats. Repeated state estimation and uncertainty-aware value update can reduce the effect of isolated unstable observations, while the quality and calibration of the estimator remain important for reliable boundary-potential computation.

Second, the mutation process depends on the capability and consistency of the mutator model. A weaker mutator may generate repetitive edits, violate operator constraints, or fail to preserve the intended template structure. In addition, the current evaluation mainly uses English safety-evaluation queries and a fixed set of public benchmarks. Additional experiments on multilingual queries, domain-specific safety policies, longer interaction contexts, and model updates over time would provide a more complete understanding of how defense-state transitions behave across deployment settings.

5.3 Responsible Use and Release

This work studies automated jailbreak discovery as a defensive evaluation method for improving LLM safety. The paper reports aggregate success rates, query costs, state-transition behavior, and stability statistics. It omits generated templates, mutation chains, and response excerpts that could be directly reused as operational jailbreak content. The evaluation queries come from pre-collected safety benchmark sets, and the experiments do not target real individuals, organizations, or ongoing events.

Generated logs and annotations are stored in access-controlled environments. Researchers involved in annotation or review receive warnings before exposure to potentially harmful model outputs and may opt out of related tasks. For artifact release, the recommended package includes implementation code for the search framework, defense-state estimator training. Operational prompt logs and successful generated templates should remain excluded from public release.

5.4 Implications for Safety Evaluation

DeST-Fuzzing can support model developers by providing a repeatable audit procedure for black-box safety evaluation. The transition statistics identify which defense states occur frequently, which mutation operators tend to produce progress, and where regressions appear during search. These diagnostics can guide defense tuning, adversarial test-set construction, and post-update regression testing. For research, the results suggest that automated jailbreak evaluation benefits from modeling LLM defenses as stochastic state-transition systems. This perspective encourages future work on calibrated defense-state estimators, stronger uncertainty modeling, multilingual boundary analysis, and defenses that reduce partial-acceptance states. It also provides a way to compare automated red-team systems through stable transition behavior together with final success labels.

6 Conclusion

This paper presents DeST-Fuzzing, a defense-state transition optimization framework for black-box automated jailbreak fuzzing. It maps target responses into ordered defense states, estimates boundary potential from repeated response-state observations, and models mutation operators as state-transition actions. The framework combines transition-aware operator selection, non-linear tree search, dynamic intermediate expansion, and uncertainty-gated value update. Experiments on commercial and open-source models show that DeST-Fuzzing improves query coverage and query efficiency over automated fuzzing baselines. The hard-subset, ablation, and stability analyses further indicate that boundary potential, state-conditioned operator selection, and uncertainty-aware value propagation contribute to more stable discovery. Overall, DeST-Fuzzing provides a controlled framework for evaluating LLM safety robustness and identifying weaknesses in safety alignment.

References

[1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. GPT-4 Technical Report. *arXiv preprint arXiv:2303.08774* (2023).

[2] Anthropic. 2024. Claude 3.5 Sonnet Model Card Addendum. https://www.anthropic.com/system-cards. Accessed 2026-04-30.

[3] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Alex Goldie, Azalia Mirhoseini, Chris McKinnon, et al. 2022. Constitutional AI: Harmlessness from AI Feedback. *arXiv preprint arXiv:2212.08073* (2022).

[4] Emily M Bender, Timnit Gebru, Angelina McMillan-Major, and Shmargaret Shmitchell. 2021. On the dangers of stochastic parrots: Can language models be too big?. In *Proceedings of the 2021 ACM conference on fairness, accountability, and transparency*. 610–623.

[5] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.

[6] Patrick Chao, Edoardo Debenedetti, Alexander Robey, Maksym Andriushchenko, Francesco Croce, Vikash Sehwag, Edgar Dobriban, Nicolas Flammarion, George J Pappas, Florian Tramer, et al. 2024. Jailbreakbench: An open robustness benchmark for jailbreaking large language models. *Advances in Neural Information Processing Systems* 37 (2024), 55005–55029.

[7] Patrick Chao, Alexander Robey, Edgar Dobriban, Hamed Hassani, George J. Pappas, and Eric Wong. 2025. Jailbreaking Black Box Large Language Models in Twenty Queries. In *2025 IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*. 23–42.

[8] Yongkang Chen, Chongyang Zhao, Jianwentian Jianwentian, Guiling Cao, Hu Li, and Xiaohui Kuang. 2025. Better Red Teaming via Searching with Large Language Model. In *Findings of the Association for Computational Linguistics: ACL 2025*. 4968–4984.

[9] Paul F Christiano, Jan Leike, Tom Brown, Miljan Martic, Shane Legg, and Dario Amodei. 2017. Deep Reinforcement Learning from Human Preferences. In *Advances in Neural Information Processing Systems*, Vol. 30. 1–9.

[10] Gelei Deng, Yi Liu, Yuekang Li, Kailong Wang, Ying Zhang, Zefeng Li, Haoyu Wang, Tianwei Zhang, and Yang Liu. 2024. MASTERKEY: Automated Jailbreaking of Large Language Model Chatbots. In *31st Annual Network and Distributed System Security Symposium, NDSS*.

[11] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. 2024. The llama 3 herd of models. *arXiv e-prints* (2024), arXiv–2407.

[12] Georgios Fatouros, John Soldatos, Kalliopi Kouroumali, Georgios Makridis, and Dimosthenis Kyriazis. 2023. Transforming sentiment analysis in the financial domain with ChatGPT. *Machine Learning with Applications* 14 (2023), 100508.

[13] Samuel Gehman, Suchin Gururangan, Maarten Sap, Yejin Choi, and Noah A. Smith. 2020. RealToxicityPrompts: Evaluating Neural Toxic Degeneration in Language Models. In *Findings of the Association for Computational Linguistics: EMNLP*, Vol. EMNLP 2020. 3356–3369.

[14] Samyak Jain, Ekdeep Singh Lubana, Kemal Oksuz, Tom Joy, Philip H.S. Torr, Amartya Sanyal, and Puneet K. Dokania. 2024. What Makes and Breaks Safety Fine-tuning? A Mechanistic Study. In *Advances in Neural Information Processing Systems*, Vol. 37. 93406–93478.

[15] Fengqing Jiang, Zhangchen Xu, Luyao Niu, Zhen Xiang, Bhaskar Ramasubramanian, Bo Li, and Radha Poovendran. 2024. ArtPrompt: ASCII Art-based Jailbreak Attacks against Aligned LLMs. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*. 15157–15173.

[16] Erik Jones, Anca Dragan, Aditi Raghunathan, and Jacob Steinhardt. 2023. Automatically Auditing Large Language Models via Discrete Optimization. In *Proceedings of the 40th International Conference on Machine Learning*, Vol. 202. 15307–15329.

[17] Diederik P. Kingma and Jimmy Ba. 2015. Adam: A Method for Stochastic Optimization. In *3rd International Conference on Learning Representations, ICLR 2015*.

[18] Andreas Köpf, Yannic Kilcher, Dimitri von Rütte, Sotiris Anagnostidis, Zhi Rui Tam, Keith Stevens, Abdullah Barhoum, Duc Nguyen, Oliver Stanley, Richárd Nagyfi, Shahul ES, Sameer Suri, David Glushkov, Arnav Dantuluri, Andrew Maguire, Christoph Schuhmann, Huu Nguyen, and Alexander Mattick. 2023. OpenAssistant Conversations - Democratizing Large Language Model Alignment. In *Advances in Neural Information Processing Systems*, Vol. 36. 47669–47681.

[19] Stephanie Lin, Jacob Hilton, and Owain Evans. 2022. Truthfulqa: Measuring how models mimic human falsehoods. In *Proceedings of the 60th annual meeting of the association for computational linguistics*. 3214–3252.

[20] Tong Liu, Yingjie Zhang, Zhe Zhao, Yinpeng Dong, Guozhu Meng, and Kai Chen. 2024. Making Them Ask and Answer: Jailbreaking Large Language Models in Few Queries via Disguise and Reconstruction. In *33rd USENIX Security Symposium*. 4711–4728.

[21] Xiaogeng Liu, Nan Xu, Muhao Chen, and Chaowei Xiao. 2024. AutoDAN: Generating Stealthy Jailbreak Prompts on Aligned Large Language Models. In *The Twelfth International Conference on Learning Representations*.

[22] Yi Liu, Chengjun Cai, Xiaoli Zhang, Xingliang Yuan, and Cong Wang. 2024. Arondight: Red Teaming Large Vision Language Models with Auto-generated Multi-modal Jailbreak Prompts. 3578–3586.

[23] Yi Liu, Gelei Deng, Zhengzi Xu, Yuekang Li, Yaowen Zheng, Ying Zhang, Lida Zhao, Tianwei Zhang, and Yang Liu. 2023. Jailbreaking ChatGPT via Prompt Engineering: An Empirical Study. *CoRR* abs/2305.13860 (2023). doi:10.48550/ARXIV.2305.13860

[24] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. RoBERTa: A Robustly Optimized BERT Pretraining Approach. *CoRR* abs/1907.11692 (2019).

[25] Mantas Mazeika, Long Phan, Xuwang Yin, Andy Zou, Zifan Wang, Norman Mu, Elham Sakhaee, Nathaniel Li, Steven Basart, Bo Li, David A. Forsyth, and Dan Hendrycks. 2024. HarmBench: A Standardized Evaluation Framework for Automated Red Teaming and Robust Refusal. In *Forty-first International Conference on Machine Learning, ICML 2024*.

[26] Anay Mehrotra, Manolis Zampetakis, Paul Kassianik, Blaine Nelson, Hyrum Anderson, Yaron Singer, and Amin Karbasi. 2024. Tree of attacks: Jailbreaking black-box llms automatically. *Advances in Neural Information Processing Systems* 37 (2024), 61065–61105.

[27] Sharan Narang, Gregory Diamos, Erich Elsen, Paulius Micikevicius, Jonah Alben, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, et al. 2018. Mixed precision training. In *International Conference on Learning Representations*.

[28] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[29] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. 2025. Asleep at the keyboard? assessing the security of github copilot's code contributions. *Commun. ACM* 68, 2 (2025), 96–105.

[30] Ethan Perez, Saffron Huang, H. Francis Song, Trevor Cai, Roman Ring, John Aslanides, Amelia Glaese, Nat McAleese, and Geoffrey Irving. 2022. Red Teaming Language Models with Language Models. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing, EMNLP*, Yoav Goldberg, Zornitsa Kozareva, and Yue Zhang (Eds.). 3419–3448.

[31] Alberto Purpura, Sahil Wadhwa, Jesse Zymet, Akshay Gupta, Andy Luo, Melissa Kazemi Rad, Swapnil Shinde, and Mohammad Shahed Sorower. 2025. Building Safe GenAI Applications: An End-to-End Overview of Red Teaming for Large Language Models. In *Proceedings of the 5th Workshop on Trustworthy NLP*. 335–350.

[32] Mark Russinovich, Ahmed Salem, and Ronen Eldan. 2025. Great, now write an article about that: The crescendo Multi-Turn LLM jailbreak attack. In *34th USENIX Security Symposium*. 2421–2440.

[33] Xinyue Shen, Zeyuan Chen, Michael Backes, Yun Shen, and Yang Zhang. 2024. "Do Anything Now": Characterizing and Evaluating In-The-Wild Jailbreak Prompts on Large Language Models. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. ACM, 1671–1685.

[34] Kazuhiro Takemoto. 2024. All in how you ask for it: Simple black-box method for jailbreak attacks. *Applied Sciences* 14, 9 (2024), 3558.

[35] Arun James Thirunavukarasu, Darren Shu Jeng Ting, Kabilan Elangovan, Laura Gutierrez, Ting Fang Tan, and Daniel Shu Wei Ting. 2023. Large language models in medicine. *Nature medicine* 29, 8 (2023), 1930–1940.

[36] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, et al. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. *arXiv preprint arXiv:2307.09288* (2023).

[37] Yifei Wang, Yuyang Wu, Zeming Wei, Stefanie Jegelka, and Yisen Wang. 2024. A Theoretical Understanding of Self-Correction through In-context Alignment. In *Advances in Neural Information Processing Systems*, Vol. 37. 89869–89912.

[38] Alexander Wei, Nika Haghtalab, and Jacob Steinhardt. 2023. Jailbroken: How Does LLM Safety Training Fail?. In *Advances in Neural Information Processing Systems*, Vol. 36. 80079–80110.

[39] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.

[40] Nan Xu, Fei Wang, Ben Zhou, Bangzheng Li, Chaowei Xiao, and Muhao Chen. 2024. Cognitive Overload: Jailbreaking Large Language Models with Overloaded Logical Thinking. In *Findings of the Association for Computational Linguistics: NAACL 2024*. 3526–3548.

[41] Jiahao Yu, Xingwei Lin, Zheng Yu, and Xinyu Xing. 2024. LLM-Fuzzer: Scaling Assessment of Large Language Model Jailbreaks. In *33rd USENIX Security Symposium*. 4657–4674.

[42] Zhiyuan Yu, Xiaogeng Liu, Shunning Liang, Zach Cameron, Chaowei Xiao, and Ning Zhang. 2024. Don't Listen To Me: Understanding and Exploring Jailbreak Prompts of Large Language Models. In *33rd USENIX Security Symposium (USENIX Security 24)*. 4675–4692.

[43] Yichi Zhang, Zihao Zeng, Dongbai Li, Yao Huang, Zhijie Deng, and Yinpeng Dong. 2025. RealSafe-R1: Safety-Aligned DeepSeek-R1 without Compromising Reasoning Capability. *arXiv preprint arXiv:2504.10081* (2025).

[44] Yiran Zhao, Wenyue Zheng, Tianle Cai, Xuan Long, Kenji Kawaguchi, Anirudh Goyal, and Michael Qizhe Shieh. 2024. Accelerating Greedy Coordinate Gradient and General Prompt Optimization via Probe Sampling. In *Advances in Neural Information Processing Systems*, Vol. 37. 53710–53731.

[45] Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, J. Zico Kolter, and Matt Fredrikson. 2023. Universal and Transferable Adversarial Attacks on Aligned Language Models. *arXiv preprint arXiv:2307.15043* (2023).

Appendix

A Open Science

A.1 Artifact Location and Review Access

The artifacts for double-blind review will be made available at the following anonymous artifact location:

https://anonymous.4open.science/r/DeST-Fuzzing-FB7F/

The URL above should be the same as the URL submitted in HotCRP. The artifact repository is intended to be frozen by the online artifact deadline and should remain unchanged throughout the review period. The repository should be hosted on an anonymous artifact service that does not reveal author identity and does not provide authors with reviewer-identifying access logs.

A.2 Artifact Inventory

Table 8 summarizes the artifacts needed to evaluate the main contributions of DeST-Fuzzing. The release plan separates artifacts that are necessary for methodology and reproducibility from artifacts that are withheld because they would directly enable operational jailbreak reuse. To balance reproducibility and responsible release, we provide the implementation code, configuration templates, initial evaluation queries, initial prompt templates, mutation/evaluation prompts, and execution instructions required to rerun the framework. We do not publicly release generated records, intermediate logs, cached outputs, mutation trajectories, or target-model responses, because these files may contain enough information to reconstruct directly executable malicious prompts. Reviewers can regenerate the corresponding records by running the released framework with their own API credentials or local model endpoints.

A.3 Reproducibility Scope

The artifact supports two levels of reproducibility. First, the synthetic benchmark should allow reviewers to verify that initialization, mutation selection, state estimation, tree update, and metric computation execute end to end. Second, reviewers with their own API access should be able to rerun the black-box experiments under the same budget-accounting rules. Because commercial model behavior and provider-side safety filters may change over time, reruns may not exactly reproduce historical target responses.

A.4 Artifacts That Cannot Be Fully Shared

The raw operational prompt artifacts cannot be fully shared in an open review bundle. This includes successful generated templates, mutation chains that lead to state 3, exact target-model response excerpts, and any raw logs that combine a malicious query with a high-performing template. Releasing those artifacts would materially increase misuse risk by providing reusable jailbreak content. The submission therefore uses a capability-reduced release: code, configurations are available, while operational prompt content is withheld.

B Ethical Considerations

B.1 Risk-Benefit Assessment

This work studies automated jailbreak discovery for defensive security evaluation. The primary benefit is to provide model developers and safety auditors with a repeatable way to identify weak regions of LLM safety alignment, measure the stability of discovered failures, and construct regression tests after model updates. The main risk is dual use: an automated fuzzing framework, successful templates, or detailed mutation traces could be repurposed to generate harmful content or to improve offensive jailbreak attempts. The paper and artifact release are therefore designed to report aggregate scientific evidence while withholding operational content that could be directly reused.

B.2 Misuse Mitigations

The submission uses the following safeguards.

(1) It reports aggregate success rates, query costs, transition statistics, and stability metrics rather than publishing successful jailbreak templates or target-model response excerpts.
(2) It excludes raw generated templates, raw mutation chains, and operational prompt logs from the anonymous artifact bundle.
(3) It includes only synthetic or benign examples for end-to-end artifact execution.
(4) It treats all generated harmful responses and annotations as sensitive data and stores them in access-controlled environments.
(5) It recommends controlled research-only access for any sensitive materials that may be needed after review, subject to institutional identity verification, approved-use terms, and non-redistribution commitments.

B.3 Human Subjects, Annotation, and Data Handling

The experiments do not target real individuals, organizations, or ongoing real-world events. Evaluation queries are drawn from pre-collected safety benchmark sources and are used only for controlled robustness measurement. The study does not collect private user data. Human involvement is limited to annotation or review of model outputs for defense-state estimation and validation. Annotators should receive a safety notice before exposure to potentially harmful model outputs, may opt out of such tasks, and should not be asked to provide personal or demographic information as part of the annotation data. Regarding governance and disclosure, the study undergoes Institutional Review Board (IRB) review and is deemed not to involve human subjects, meeting relevant ethical and compliance requirements.

B.4 Responsible Evaluation of Target Models

The black-box experiments are framed as security auditing rather than service disruption. The evaluation uses ordinary prompt interfaces and bounded query budgets, and does not attempt to bypass authentication, payment, rate limits, or access controls. API credentials are not shared in the artifact. If a concrete provider-specific

Table 8: Artifact inventory for the Open Science appendix.

Artifact	Review availability	Purpose and notes
Search framework implementation	Included.	Implements the proposed tree search, transition-aware operator selection, uncertainty-gated update, budget accounting, and output ranking.
Defense-state estimator implementation	Included.	Provides the code and interfaces for mapping target-model responses to the four defense states.
Configuration templates	Included.	Provides reusable settings for models, decoding parameters, query budgets, sampling counts, tree-search parameters, random seeds, and metrics.
Initial evaluation queries	Included.	Provides the initial malicious evaluation questions used to instantiate the safety-audit tasks.
Initial prompt templates	Included.	Provides the initial seed templates used to initialize the search process.
Mutation and evaluation prompts	Included.	Provides the instruction templates used by the mutator, evaluator, and auxiliary components.
Mutation operator implementation	Included.	Provides the implementation of template-level mutation operators and their input-output interfaces.
Evaluation and metric scripts	Included.	Computes JQN, Top-1 ASR, Top-5 ASR, AvgQ, Stable ASR, Reproduction Rate, transition diagnostics, and sensitivity summaries from newly generated runs.
Run instructions and documentation	Included.	Describes installation, credential configuration, experiment execution, and metric computation.
Commercial-model API credentials	Not included.	Reviewers should use their own API credentials or local model endpoints.
Generated records and logs	Not publicly released.	Reviewers can regenerate these files by running the released framework with the provided initial inputs and configuration templates.
Generated jailbreak templates, mutation chains, and target-model responses	Not publicly released.	These generated artifacts are excluded from the public review artifact.

vulnerability is discovered that requires disclosure beyond aggregate reporting, the authors will use the provider's responsible disclosure channel and avoid publishing operational exploit details before mitigation.

C Generative AI Usage

OpenAI ChatGPT was used to assist with language-level editing of the main manuscript, including grammar correction, wording refinement, and readability improvement. It was also used to help draft parts of the appendix. The tool was not used as an author and did not autonomously generate experimental results, tables, plots, labels, or citations. The human authors are responsible for all submitted content, including the accuracy of technical claims, artifact descriptions, ethical statements, and references.

D Threat Model and Review-Scope Clarification

D.1 Attacker and Auditor Capabilities

The evaluated adversary is a black-box red-team auditor who can submit textual prompts to a target LLM and observe generated responses. The adversary can maintain a seed library of prompt templates, use a mutator model to propose template edits, and allocate a bounded number of target-model calls. The adversary does not have access to target-model weights, gradients, logits, hidden states, training data, system prompts, private safety-filter rules, or provider-side moderation internals.

D.2 Threat Surface

The threat surface is the natural-language prompt interface of deployed or locally hosted LLMs. DeST-Fuzzing evaluates whether prompt templates can move target responses from refusal states toward partial or full acceptance states under repeated stochastic generation. The work does not study model extraction, training-time poisoning, account compromise, data exfiltration, API abuse, or attacks on infrastructure surrounding the model service.

D.3 Generality and Practicality

The method is intended to be model-agnostic under a black-box API setting. Its transition statistics are estimated from observed responses rather than from model internals. The evaluation includes both commercial and open-source targets and uses a fixed query budget, repeated state estimation, and consistent metric definitions. Practicality is limited by the cost of repeated target-model calls and by the quality of the mutator model and defense-state estimator; these limitations are addressed through budget accounting and uncertainty penalties.

D.4 Defender Objective

The defender's objective is to identify unstable or weak safety-boundary regions before misuse occurs, to compare model robustness under a common audit protocol, and to support regression testing after model or policy updates. The framework should be

Table 9: Mutation operator descriptions. The table summarizes template-level editing behaviors without exposing operational jailbreak prompts.

Operator	Group	Intended transformation
Expand	Radical exploration	Adds high-level context, framing, or task-independent scaffolding to increase semantic diversity and escape refusal-dominated regions.
Crossover	Radical exploration	Combines abstract structural elements from two parent templates to produce a new template skeleton.
Style Transfer	Radical exploration	Changes tone, genre, or presentation style while preserving the abstract template role.
Rephrase	Conservative infiltration	Rewrites local wording, syntax, or ordering while preserving the semantic skeleton of the parent template.
Shorten	Conservative infiltration	Removes redundancy and compresses the template to reduce unnecessary surface features.
Polish	Conservative infiltration	Improves fluency, grammar, and coherence without changing the template skeleton.

used for authorized safety evaluation and not for generating or distributing operational jailbreak prompts.

E Operator Descriptions and Implementation Constraints

This appendix provides the operator descriptions and implementation constraints referenced in Section 3.1. The descriptions below are intentionally capability-reduced: they specify transformation goals and safety constraints, but do not include reusable jailbreak templates, successful mutation chains, or operational prompt text.

E.1 Common Operator Interface

Each mutation operator receives a parent template T, the operator identifier a, and a structured set of constraints. The operator returns a candidate child template T'. The malicious query Q is not embedded into the operator prompt during template editing unless required by the experimental implementation; when included, it is treated as a placeholder rather than copied into released artifacts.

E.2 Operator Cards

Table 9 describes the six mutation operators used in DeST-Fuzzing. Radical exploration operators are designed to make larger structural changes, while conservative infiltration operators perform local refinements near the defense boundary. In the implementation, all operators remain available for expandable states; the transition-aware policy selects among them using the state-conditioned utility and the exploration bonus.